PORTFOLIO

INTRODUCE

오준석

BackEnd Developer

Contact.

Email. work.ditto909@passmail.net

Phone. 010-6657-2531

Channel.

Resume. junseokhub.github.io

GitHub. github.com/junseokhub

Experience

Gentlepie

2022.04 ~ 2025.05 (BackEnd Deplopment)

PROJECTS

Bublitt

2023.09 ~ 2025.05 | Spring Boot, Milvus, Kubernetes, ArgoCD, Kafka

🔍 프로젝트 배경
다양한 고객사의 챗봇을 하나의 플랫폼에서 관리하는 AI 챗봇 빌더. 초기 NestJS 기반 서비스를 Spring Boot로 전환하며 AI 응답 품질, 동시성, 비동기 처리까지 전반적인 시스템 고도화를 주도.

📊 성과

- RAG 아키텍처 + ReRank 모델 도입으로 응답 정답률 65.24% → 93.71%
- DBSCAN 클러스터링 기반 주제 추천 시스템으로 응답 신뢰도 20% 향상
- Jenkins CI → ArgoCD GitOps 전환으로 배포 실패율 30% 감소
- 온프레미스 서버 구축으로 인프라 비용 50% 절감

⚙️ 기술적 의사결정

🔹 RAG 응답 정답률 개선

[문제]
단순 벡터 유사도 검색만으로는 의미적으로 유사하지만 실제로는 관련 없는 문서가 상위에 노출되는 문제 발생

[원인]
Embedding 모델 한계 + 데이터 전처리 미흡으로 벡터 공간에서 노이즈 데이터 혼재

[해결]
데이터 전처리 방식 개선 + ReRank 모델 도입으로 검색 결과 재정렬 → 정답률 65.24% → 93.71% 달성

🔹 NestJS → Spring Boot 마이그레이션 동시성 문제

[문제]
마이그레이션 후 Token 과금, 벡터 DB 업데이트 등 동시 요청 시 데이터 정합성이 깨지는 문제 발생

[원인]
NestJS는 싱글스레드 이벤트루프 기반이라 애플리케이션 레벨 동시성 충돌이 없었지만 Spring Boot 멀티스레드 환경에서는 DB 레벨 충돌 발생

[해결]
Optimistic Lock + Pessimistic Lock 이중 전략 + 재시도 로직으로 멀티스레드 환경 데이터 정합성 확보

🔹 I/O 블로킹 문제

[문제]
OpenAI Embedding → Milvus 검색 → OpenAI Chat Completion 3개의 외부 API 순차 호출로 서블릿 스레드가 응답 대기 시간 동안 점유되어 처리량 저하

[원인]
RAG 파이프라인 특성상 3단계가 순차 의존 관계라 병렬화 자체가 불가능한 구조

[해결]
WebFlux 전환 검토했으나 MVC 레거시 전환 비용이 효과 대비 높다고 판단, CompletableFuture + 전용 ioExecutor 분리로 서블릿 스레드 해방 + Timeout 처리 적용

⚠️ 한계 및 향후 방향

DBSCAN 클러스터링 불안정
매주 새로운 데이터가 입력되면서 밀도가 지속 변동, eps 파라미터 최적값이 일관되지 않아 노이즈 과다 또는 관련 없는 데이터가 하나의 군집으로 병합되는 문제 발생
→ HDBSCAN 도입 검토 중, eps 없이 밀도 변동에 유연하게 대응 가능할 것으로 예상

비동기 처리 한계
CompletableFuture는 블로킹을 다른 스레드로 위임하는 구조라 트래픽 급증 시 스레드 풀 고갈 위험 존재
→ Spring MVC 유지하면서 무거운 작업을 Kafka 이벤트 기반으로 분리하는 방향으로 전환 검토 중

🧩 마무리
단순히 동작하는 시스템을 넘어 왜 이 구조인지, 어떤 한계가 있는지를 끝까지 파고드는 과정에서 기술적 의사결정과 트레이드오프를 고민하는 습관을 만든 프로젝트.

Coconutive

2023.10 ~ 2025.05 | Nest.js, Redis, MySQL, MongoDB
🔍 프로젝트 배경

챗봇 사용 데이터를 수집·분석하는 통계 솔루션. Raw SQL 기반 레거시 코드를 점진적으로 개선하며 운영 안정성과 데이터 관리 체계를 구축.

📊 성과

- 중복 로그인 세션 문제 32% → 0% 해결
- Raw SQL → TypeORM 전환 70%+ 완료로 유지보수성 향상
- 자연어 통계 엔진 → LLM 기반 전환으로 분류 자동화 및 정확도 향상
- 수동 로그 관리 → MongoDB CronJob 자동화로 운영 안정성 확보

⚙️ 기술적 의사결정

🔹 Raw SQL → TypeORM 전환 전략

[문제]
고객사별로 DB 환경이 달라 LIMIT/OFFSET, 날짜, JOIN 최적화 문법이 조금씩 달라 DB별 쿼리 유지보수 부담 지속 증가

[원인]
초기 Raw SQL로 빠르게 개발하다 보니 DB 종속적인 쿼리가 전체에 혼재

[해결]
100% 전환 시도했으나 다중 JOIN, 동적 조건 필터, 집계 함수 등 복잡한 쿼리는 TypeORM이 오히려 가독성이 떨어진다고 판단
→ 단순 CRUD/동적 필터 → TypeORM
→ 복잡한 통계/집계 쿼리 → Raw SQL 유지
기준을 명확히 정의하여 70%+ 전환 완료

🔹 Redis 세션 관리 도입

[문제]
중복 로그인으로 인한 세션 충돌 문제 32% 발생, 보안 및 데이터 일관성 저하

[해결]
Redis 중앙 집중형 세션 관리 도입으로 중복 로그인 제어 및 세션 만료 정책 적용
→ 중복 로그인 문제 32% → 0% 해결

⚠️ 한계 및 향후 방향

Raw SQL 완전 제거의 한계 복잡한 통계 쿼리는 TypeORM QueryBuilder보다 Raw SQL이 가독성·성능 면에서 현실적으로 우위.
→ 남은 Raw SQL은 DB별로 SQL 분리 + Executor 패턴으로 런타임에 DB 종류 확인 후 해당 SQL 로딩하는 방향 검토 중

🧩 마무리
모든 코드를 한 번에 바꾸는 것보다 비즈니스 연속성을 유지하며 점진적으로 개선하는 것이 엔지니어링의 본질임을 체감한 프로젝트.

식약처 코스봇

2024.05 ~ 2025.05 | Spring Boot, Milvus, Kubernetes

🔍 프로젝트 배경

기존 분류형 챗봇의 정보 탐색 난이도가 높고 기계적인 말투로 사용자 만족도가 낮아 Bublitt 플랫폼 기반 Upstage RAG(Solar) 도입으로 식약처 맞춤형 LLM 챗봇으로 전환. (기여도 100%)

📊 성과

- Embedding 모델·벡터 차원별 응답 정확도 비교 테스트 모듈 개발로 최적 모델 선정 체계 구축
- 기관 맞춤형 통계 대시보드로 의사결정 데이터 시각화 제공
- Prometheus + Grafana 모니터링 + ArgoCD GitOps 배포로 운영 안정성 확보

⚙️ 기술적 의사결정

🔹 Embedding 모델 선정 체계 구축

[문제]
모델마다 응답 정확도가 다르고 벡터 차원에 따라 검색 성능이 달라 어떤 조합이 최적인지 기준이 없었음

[해결]
모델별·벡터 차원별 응답 정확도를 자동으로 비교하는 테스트 모듈 개발
→ 데이터 기반으로 최적 모델 선정 가능한 검증 체계 구축

🔹 기관 특화 통계 시스템

[문제]
일반적인 챗봇 통계로는 식약처 특성에 맞는 인사이트 도출이 어려움

[해결]
식약처 업무 패턴에 맞춘 통계 로직 설계 + 대시보드 개발로 정책 의사결정에 필요한 데이터 시각화 제

공

⚠️ 한계 및 향후 방향

Bublitt 플랫폼 의존성
핵심 RAG 로직이 Bublitt에 있어 식약처 특화 커스터마이징에 한계 존재
→ 향후 도메인 특화 Fine-tuning 또는 독립적인 RAG 파이프라인 구축 검토 중

🧩 마무리
플랫폼 연동 프로젝트에서도 기관 맞춤형 검증 체계와 통계 시스템을 직접 설계하며 도메인 이해가 기술 구현만큼 중요하다는 것을 체감한 프로젝트.

SK-EcoPlant 사내 챗봇

2023.04 ~ 2025.05 | Nest.js, TypeORM, DialogFlow CX, AWS

🔍 프로젝트 배경

사내 포털에서 직원 불편접수가 SNS로 퍼지며 운영팀 피로도가 높아지는 문제를 해결하기 위해 대화형이 아닌 정보형 챗봇으로 접수 채널을 전환. 백엔드 및 인프라 전 영역 단독 설계. (기여도 100%)

📊 성과

- DialogFlow CX Slot Filling으로 대화 시나리오별 정보 자동 수집 및 문맥 기반 응답 구현
- 어드민에서 Intent/Entity 등록·수정·삭제 가능한 관리 API 개발로 운영팀 자체 관리 체계 구축
- AWS S3 + CloudFront 기반 챗봇 SDK 배포 자동화
- Jenkins CI/CD + Load Balancer 기반 운영 환경 구축

⚙️ 기술적 의사결정

🔹 대화형 vs 정보형 챗봇 선택

[문제]
직원들이 불편사항을 SNS에 게시하는 문화가 형성되어 운영팀 피로도 증가 및 부정적 분위기 확산

[의사결정]
자유로운 대화형 챗봇은 오히려 불만을 증폭시킬 수 있다고 판단, 정해진 시나리오로 접수를 유도하는 정보형 챗봇으로 방향 결정
→ DialogFlow CX Slot Filling으로 필요한 정보를 단계적으로 수집

🔹 스프레드시트 실시간 동기화 문제

[문제]
고객사 요청으로 접수 데이터를 구글 스프레드시트에 실시간 반영했으나, 담당자가 실수로 행/열을 삭제·수정 시 데이터 복구 불가 문제 발생

[해결 과정]
폴링 방식으로 해결 시도했으나 데이터 변경이 없는 상황에서도 동기화가 발생해 리소스 낭비 문제 확인
→ 스프레드시트 편집 권한 최소화로 단기 보호 조치
→ 근본적 해결을 위해 어드민 페이지 구축으로 방향 전환, 사용자 편의성보다 데이터 정확도·보안이 우선이라 판단

⚠️ 한계 및 향후 방향

외부 도구 의존성 리스크 스프레드시트처럼 외부 도구에 의존하는 구조는 사용자 실수로 인한 데이터 손실 위험이 항상 존재
→ 초기부터 어드민 페이지로 설계했다면 더 안정적이었을 것이라는 교훈

🧩 마무리
기술적 구현보다 비즈니스 문제를 먼저 정의하고, 도구 선택이 운영 안정성에 미치는 영향을 직접 체감한 프로젝트.

운영 업무

신세계 면세점 상담톡 / 삼성닷컴 챗봇 (운영및 기능 개발)

2022.04 ~ 2023.08 | Vue.js, Express.js, OracleDB, DialogFlow ES, IBM Watson

🔍 프로젝트 배경

입사 초기 실서비스 운영을 담당하며 레거시 코드 개선과 신규 기능 개발을 병행

빠른 이슈 대응과 코드 품질 개선을 통해 서비스 안정성 확보

📊 성과

- 3개월 내 120건 이상 이슈 신속 처리로 서비스 안정성 확보
- 중복 함수·조건문 제거 및 모듈화로 레거시 코드 리팩토링, 신규 기능 추가 비용 절감
- AWS Cognito MFA 적용으로 어드민 2단계 인증 보안 강화
- Docker 배포 시 환경변수 주입 방식 적용으로 보안 취약점 해결

⚙️ 기술적 의사결정

레거시 코드 리팩토링 전략

[문제]
장기 운영 프로젝트 특성상 중복 함수, 조건문 과다로 신규 기능 추가 시 사이드 이펙트 파악이 어려운 상태

[해결]
한 번에 전체를 바꾸는 대신 기능 단위로 중복 제거 및 모듈화 진행
→ 서비스 중단 없이 점진적 개선, 신규 기능 추가 비용 절감

🧩 마무리
실서비스 운영을 처음 경험하며 빠른 이슈 대응과 레거시 개선을 병행하는 것이 얼마나 중요한지 체감한 프로젝트이며, 서비스를 멈추지 않으면서 개선하는 습관을 이때부터 만들었음

- **삼성닷컴 2024.12.31부로 서비스 종료**